

COS214 Practical Assignment 1

Caleb Jennings - u25173805

Chloe Larsen - u25004141

Task 1

1.1

Movement:

It has been labelled as *Strategy* when it should be labelled as the *State* pattern.

Routes:

It has been labelled as *State* when it should be labelled as the *Strategy* pattern.

Map:

It has been labelled as *Composite* which is correct however the implemented structure is incorrect.

Place Features:

It has been labelled as *Decorator* which is correct. However, the way it interacts with the composite Map is incorrect.

Biomes:

It has been labelled as *Abstract Factory* which is correct. However, the fact that the products are created with a switch is incorrect. Instead, there should be concrete factories that each create different types of concrete products.

The fact that the make() function returns a void pointer could imply that the current pattern is trying to be a template method.

1.2

Movement - *State*:

Context: Traveller

State: TravelMode

ConcreteState: OnFootState, OnBicycleState, InCarState, InAirState

Routes - *Strategy*:

Context: Trip

Strategy: RouteStrategy

ConcreteStrategy: ShortestRoute, FastestRoute, ScenicRoute, CheapestRoute

Map - *Composite*:

Client: GameManager

Component: MapComponent

Leaf: Location

Composite: Region

Place Features - *Decorator*:

Component: Map

ConcreteComponent: Region

Decorator: Location

ConcreteDecorator: Location derived classes

Biomes - *Abstract Factory*:

Abstract Factory: WorldBuilder

Concrete Factory's

AbstractProduct

ConcreteProduct's

1.3

#	Fault	Rule Broken	Consequence
1	Swapped pattern labels (<i>MoveStrategy</i> and <i>RouteState</i>)	Wrong pattern choice	Forces dynamic state transitions to be modeled as static algorithm injections and vice-verse
2	<i>WorldBuilder</i> uses a single <i>make()</i> method with an if/else chain	Open/Closed principle is violated & <i>Abstract factory</i>	Implements the <i>make()</i> function so that if more terrains are added then the function has to be modified. Therefore breaking the open/closed principle
3	<i>MoveStrategy</i> UML contains <i>{big switch}</i>	Violates UML notation rules	The inclusion of code in UML diagram violates UML notation rules
4	<i>Map</i> contains <i>location:vector</i> and <i>regions:vector</i> which are concrete	Violates composite pattern rules	Prevents uniform recursion and clients cannot treat leafs and composites identically
5	Lack of virtual destructors across base classes	C++ polymorphic resource ownership	Can cause severe memory leaks when deleting derived objects

6	<i>GameManager</i> connects via direct association/composition to every single concrete class and pattern	Excessive coupling	Any change anywhere in the application forces full recompilation and can break <i>GameManager</i>
7	Aggregation (◇) and Composition (◆) diamonds drawn with missing delete cleanup.	Ownership Rules	Dynamic memory allocations are abandoned on deconstructions causing memory leaks
8	Combination class explosion decorators, <i>WeatherTollLocation</i> and <i>WeatherTollQuestionLocation</i>	Open/closed principle	Subclasses per combinations means that it is impossible to stack features dynamically at runtime

1.4

The central class is *GameManager*. Design patterns exist to keep a program modular - which is in line with the concept of *separation of concerns*. Having true modular subsystems all working individually but collectively together makes the program easier to maintain because changing a piece of code somewhere in the app will not require the entire system to be refactored. This also makes the system more scalable. If everything were intertwined then the system would be difficult to modify and expand because the developer would have to be thinking about a lot more at once.

The current system architecture can be refactored to be more modular, using proper design patterns:

- Traveller should own MoveStrategy
- Map should own WorldBuilder
- QuestLocation should not own anything, it is simply a concrete decorator
- TollLocation should not use Location, it is simply a concrete decorator
- GameManager should own Trip
- Trip should own RouteState, assigned by GameManager

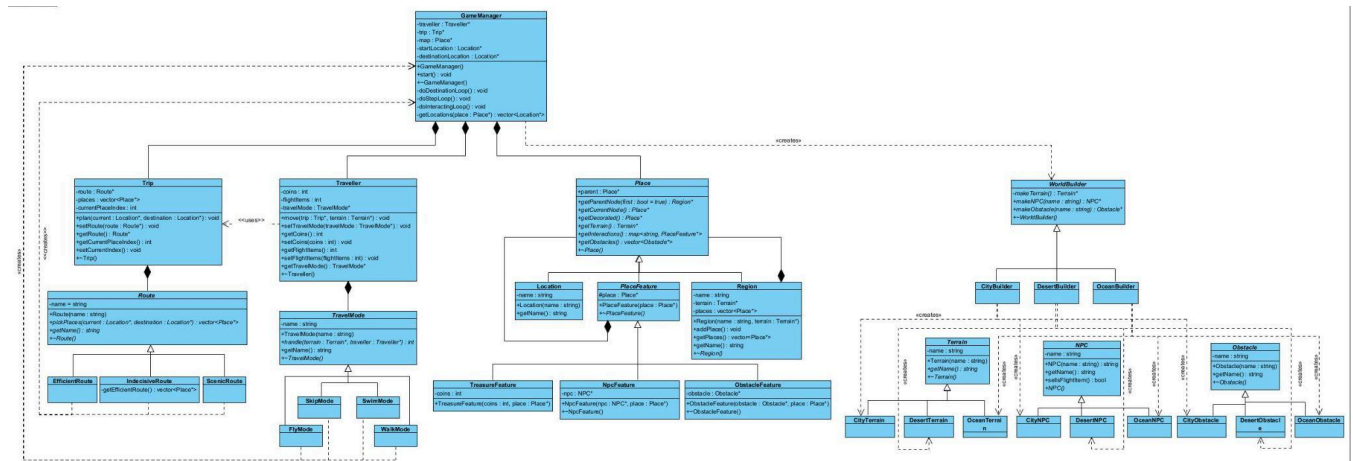
1.5

I would do the following to improve the diagram's layout:

- Prevent all relationship lines from overlapping class diagrams
- Make all relationship lines either completely horizontal or vertical
- Merge relationship lines between a parent class and derived classes of the same type - e.g. Location and Region, connect both lines with a single hollow diamond pointing into Map
- Put all secondary main classes in a line horizontally (only one not currently like this is RouteState)

Task 2

2.1



2.2

- AbstractFactory
 - Participants
 - AbstractFactory: WorldBuilder
 - ConcreteFactory: CityBuilder, DesertBuilder, OceanBuilder
 - AbstractProduct: Terrain, NPC, Obstacle
 - ConcreteProduct: CityTerrain, DesertTerrain, OceanTerrain, CityNPC, DesertNPC, OceanNPC, CityObstacle, DesertObstacle, OceanObstacle
 - Abstraction decision: I chose to make each product type abstract so that each product type can have specific implementations per biome using derived classes.
- Composite
 - Participants
 - Component: Place
 - Composite: Region
 - Leaf: Location
 - Ownership decision: I chose to make Region hold/own many Place's so that regions can recursively hold more regions or places which allows for a tree structure.
- Decorator - Overlaps with Composite
 - Participants
 - Component: Place
 - ConcreteComponent: Region, Location
 - Decorator: PlaceFeature
 - ConcreteDecorator: TreasureFeature, NpcFeature, ObstacleFeature

- Abstraction decision: The decorator is abstract so that there can be different types of decorators (concrete decorators) that each decorate in a different way, i.e. they add on additional functionality in different ways.
- State
 - Participants
 - Context: Traveller
 - State: TravelMode
 - ConcreteState: WalkMode, SkipMode, SwimMode, FlyMode
 - Ownership decision: Traveller owns/holds a travel mode to represent its current internal state.
- Strategy
 - Participants
 - Context: Trip
 - Strategy: Route
 - ConcreteStrategy: NeighboringRoute, SocialRoute, IndecisiveRoute
 - Ownership decision: Trip owns/holds a strategy to dynamically defer/swap out route selection at runtime.

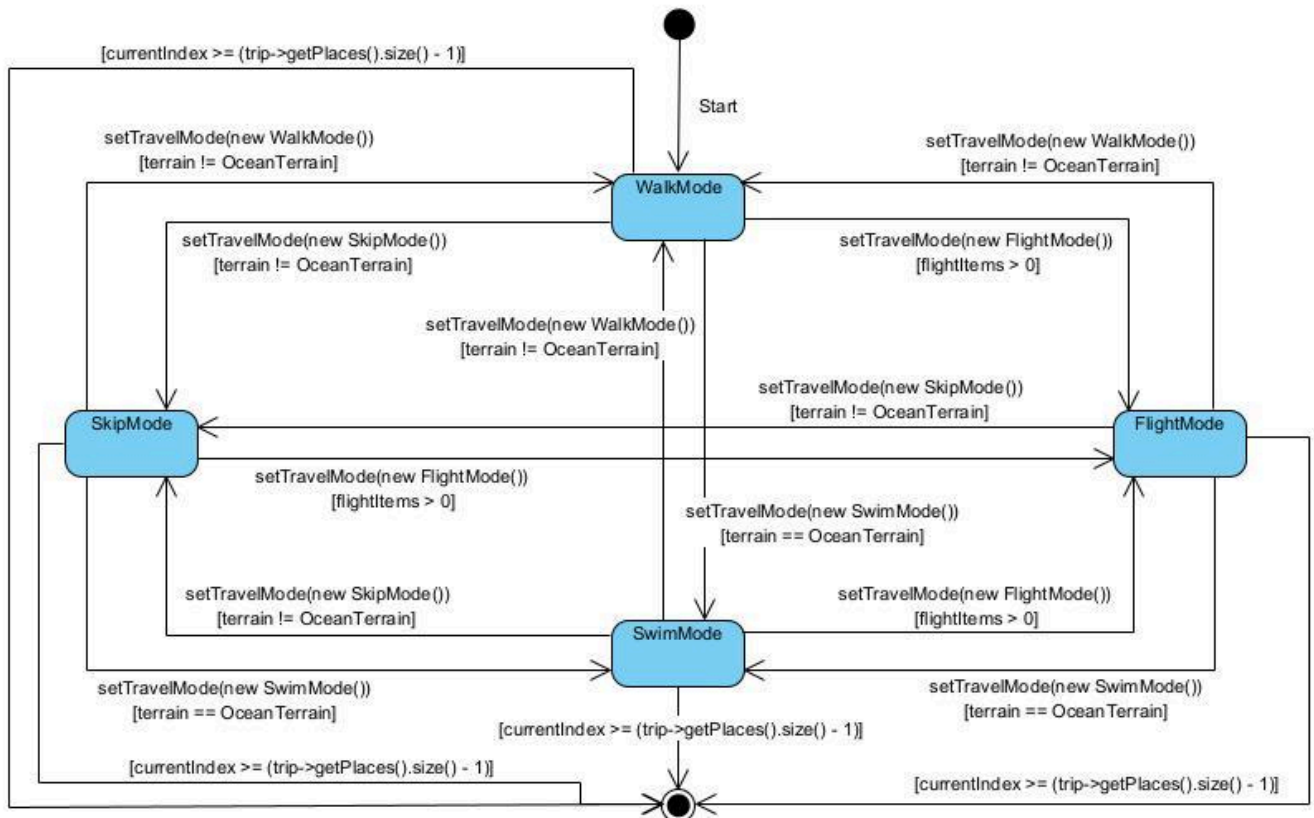
2.3

Some new features:

- Fly travel mode
 - Lets the user get to the next place much quicker, but requires a special item that must be purchased from certain NPCs.
 - Pattern: State
 - Fits easily into the pattern because all that was required to add it was creating a new class that inherits from state, and adding its own unique functionality in overridden methods.
- Obstacle place feature
 - Adds a chance the user can run into an obstacle when visiting a region that's a descendant of the ObstacleFeature in the tree
 - Pattern: Decorator
 - Fits easily into the pattern because all it needed was a new class that inherits from the decorator (PlaceFeature), with its own implementations for getInteractions() and getObstacles().
- Social route strategy
 - When selected, it picks a route that goes through places that have NPCs.
 - Pattern: Strategy
 - Fits easily into the pattern because it just requires creating a new class that inherits from Route and has its own implementation of pickPlaces() so that it can be passed to and used by Trip.

Task 3

3.3



Task 4

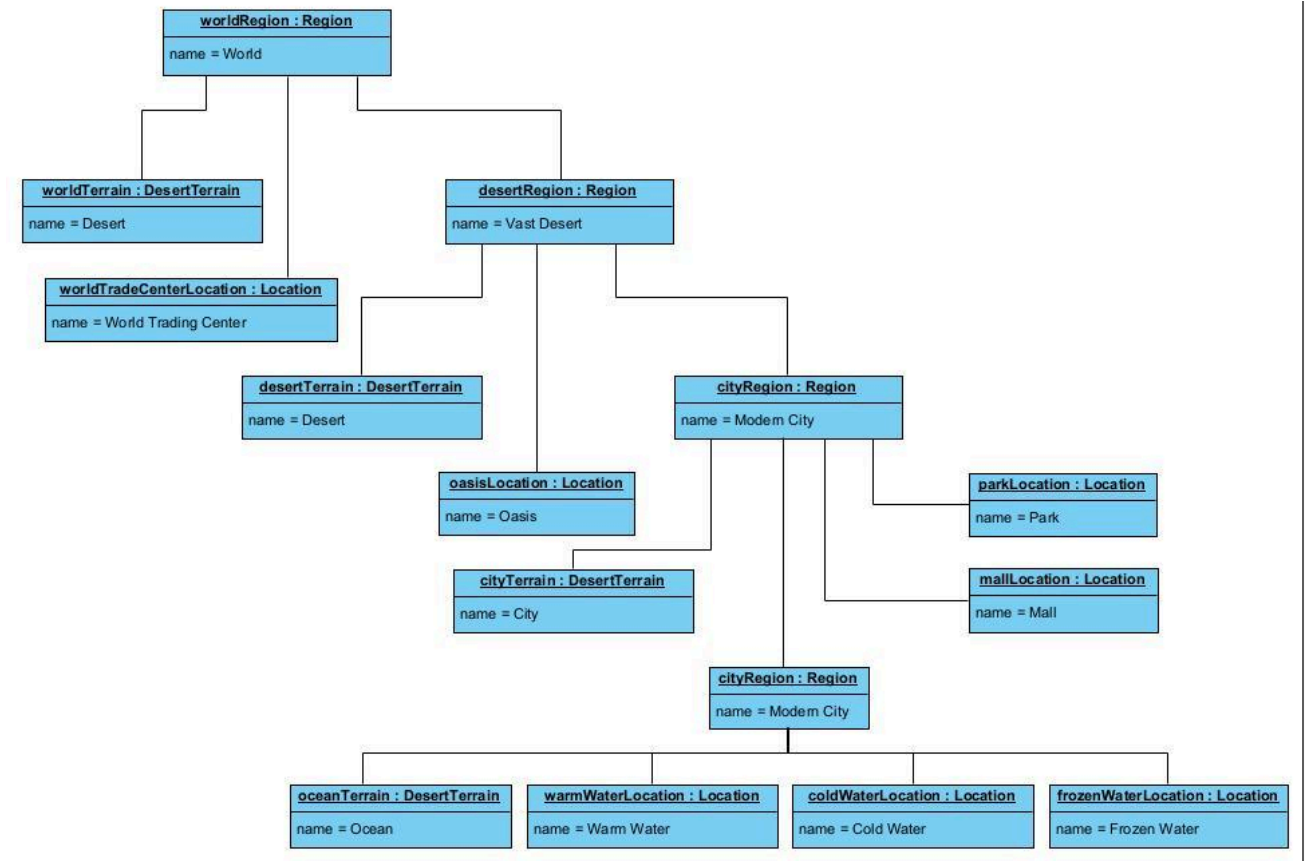
4.3

The intent of the Strategy pattern is to "Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it". While the intent of the State pattern is to "Allow an object to alter its behaviour when its internal state changes. The object will appear to change its class". So the use of Strategy pattern in the case of the routes enforces that the route is a deliberate choice made by a traveller therefore the route algorithm is self-contained and does not depend on the internal state of the traveler.

These methods are not interchangeable as in the strategy pattern the client will decide what algorithm to use and the client also holds control of when the algorithm will change. While in the state pattern the context's internal situation will determine the behavior and the states themselves control when transitions happen.

Task 5

5.2



Task 6

6.2

